

A Novel Heterogeneous Scheduling Algorithm with Improved Task Priority

Guan Wang^{1,2}, He Guo¹, Yuxin Wang^{3,*}

¹ School of Software Technology, Dalian University of Technology, Dalian, China

² Liaoning Police College, Dalian, China

³ School of Computer Science and Technology, Dalian University of Technology, Dalian, China
wyx@dlut.edu.cn

Abstract—Efficient application scheduling algorithms are important to obtain high performance in heterogeneous computing systems. However, most of current algorithms are of low efficiency in scheduling. Aiming at this problem, we propose a heterogeneous scheduling algorithm with improved task priority (HSIP) whose functionality relies on two pillars: 1) a task priority strategy based on standard deviation and Communication Cost Weight to make scheduling priority more efficient; 2) an entry task duplication selection policy to make the makespan shorter and improve the idle time slot insertion-based. We evaluate our proposed scheme on some real application DAGs by comparison with Predict Earliest Finish Time (PEFT), SD-Based Algorithm for Task Scheduling (SDBATS), Heterogeneous Earliest Finish Time (HEFT) and Critical Path On a Processor (CPOP). Experimental results show that our proposed scheme performs better than other algorithms in terms of schedule length ratio, speedup and efficiency.

Index Terms—DAG scheduling, heterogeneous computing, random graphs generator, task priority, static scheduling

1. INTRODUCTION

A heterogeneous computing system (HCS) is defined as high-speed network interconnection of multiple processors computing platform, can carry out the intensive of parallel and distributed computing. The efficiency of executing parallel applications on heterogeneous systems depends on the methods used to schedule the tasks of a parallel application. The objective is to minimize the overall

completion time (makespan) [2]. The DAG (directed acyclic graph) model is usually used for the research of task scheduling algorithm and performance evaluation. The objective of task scheduling is to map the tasks on the processors and order their execution so that task precedence requirements are satisfied and a minimum overall completion time is obtained. The problem of scheduling of tasks with required precedence relationship has been proven to be a NP-complete in the most general case [1]. Because of its key importance on performance, the task scheduling problem has been extensively studied [2-8].

Because of its low algorithm complexity as $O(v^2, p)$ and producing relative shorter scheduling length, HEFT algorithm [2] becomes the most popular and widely used algorithm. But it is considered not reasonable enough for the heterogeneous environments with different processor computing time and different communication costs between child nodes. CPOP algorithm [2] also has a complexity of $O(v^2, p)$ and in this approach all critical path tasks are assigned to the same processor, which could cause load unbalance of processors and increase the schedule length. SDBATS algorithm is based on the HEFT algorithm and make the performance significantly improved [3], but it can cause the unfairness of task scheduling. PEFT algorithm [5] is mainly focused on the subtasks of the shortest path, ignoring the larger degree of heterogeneous task scheduling priority issues.

In this paper, we present a new efficient algorithm

called Heterogeneous Scheduling with Improved Task Priority (HSIP), which outperforms state-of-the-art algorithms such as HEFT [2], CPOP [2], SDBATS [3] and PEFT [5], in terms of makespan and efficiency, and without increasing the complexity.

2. TASK-SCHEDULING PROBLEM

A scheduling system model is composed of an application, a target computing environment, and performance criteria for scheduling. An application can be represented by a directed acyclic graph, $G = (V, E)$, as shown in Figure 1, where V is the set of v nodes and each node $v_i \in V$ represents an application task. E is the set of e communication edges between tasks, each $e(i,j) \in E$ represents the task dependence constraint such that task v_i should complete its execution before task v_j can be started. The DAG is complemented by a matrix W that is a $v \times p$ computation cost matrix, where v is the number of tasks and p is the number of processors in the system. W_{ij} gives the estimated time to execute task v_i on machine p_j .

Each edge $e(i,j) \in E$ is associated with a nonnegative weight c_{ij} representing the communication cost between the tasks v_i and v_j . When both v_i and v_j are scheduled on the same processor, c_{ij} becomes zero[2].

Next, we present some of the common attributes used in task scheduling, which we will refer to in the following sections.

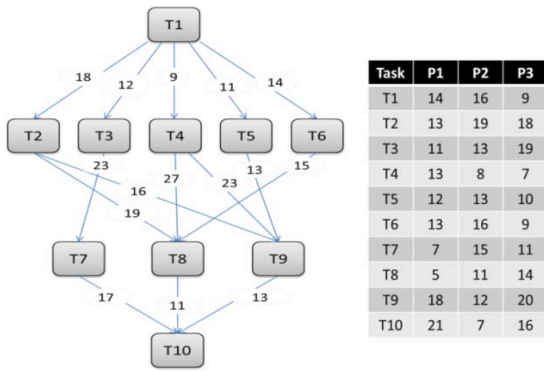


Figure 1. Example of DAG task model and computation cost matrix

Definition 1. Makespan or schedule length denotes the finish time of the last task in the scheduled DAG and is defined as equation (1) where $AFT(v_{exit})$ denotes the Actual Finish Time of the exit node.

$$\text{makespan} = \max\{AFT(v_{exit})\} \quad (1)$$

Definition 2. $EST(v_i, p_j)$ denotes the Earliest Start Time (EST) of a node v_i on a processor p_j and is defined as equation (2). For the entry task, $EST(v_{entry}, p_j) = 0$.

$$EST(v_i, p_j) = \max\{T_{Available}(p_j), \max_{v_m \in pred(v_i)} \{AFT(v_m) + c_{m,j}\}\} \quad (2)$$

Definition 3. $EFT(v_i, p_j)$ denotes the Earliest Finish Time (EFT) of a node v_i on a processor p_j and is defined as

$$EFT(v_i, p_j) = EST(v_i, p_j) + w_{i,j} \quad (3)$$

For the entry task, $EFT(v_{entry}, p_j) = w_{v_{entry},j}$

Definition 4: Out-degree Communication Cost Weight (OCCW) of task v_i means the possible max sum of communication cost generated by v_i with its immediately successors and is defined as equation (4).

$$\begin{cases} occw(v_i) = \sum_{v_j \in succ(v_i)} c_{ij} \\ occw(v_{exit}) = 0 \end{cases} \quad (4)$$

The objective of the scheduling problem is to determine an assignment of tasks of a given DAG to processors such that the schedule length is minimized. After all nodes in the DAG are scheduled, the schedule length will be the Actual Finish Time of the exit task, as expressed by (1).

3. THE PROPOSED ALGORITHM

In this section, we introduce a new scheduling algorithm for a bounded number of heterogeneous processors, called Heterogeneous Scheduling with Improved task Priority (HSIP). The algorithm has two major phases: a task prioritizing phase for computing task priorities, and a processor selection phase for selecting the best processor for executing the current task.

3.1 Task Prioritizing Phase

In descending order of the $rank_u$ value as a scheduling priority, the upward rank, $rank_u$, of each task has been calculated using the following equation.

$$\begin{cases} rank_u(v_i) = \max_{v_j \in succ(v_i)} \{\bar{w}_i \times \sigma_i + occw(v_i) + rank_u(v_j)\} \\ rank_u(v_{exit}) = \bar{w}_{exit} \times \sigma_{exit} \end{cases} \quad (5)$$

Where σ_i is the standard deviation of computation cost of any given task v_i on the available pool of processors.

However, the standard deviation value of the calculation cost is far below the communication cost weight of task. So standard deviation multiplied by the average cost is used as the calculation cost weight. Thus, the task with larger difference of computing cost could get higher priority, and so as to the child nodes' transmission time. In fact, our algorithm $rank_u$ equation can produce better scheduling policy comparing to the other state-of-the-art algorithms, and the results as shown in Figure 2. Descending order of the upward rank $rank_u(v_i)$ is as the task priority in our

approach. The upward rank of each task for instance provided by Figure1 is shown in Table1.

TABLE 1 PRIORITY WEIGHTS OF TASKS

function	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}
$rank_u(v_i)$	335.6	233.4	209.6	229.1	182.2	184.7	137.9	133.4	154.6	85.0

3.2 Processor Selection Phase

Processor selection phase is usually according to the priority of task scheduling order, assigned to the minimum EFT processor to execute [2]. On the basis of the above strategy, we proposed two innovative policies, entry task duplication selection policy and ITS insertion-based optimizing policy, in order to improve the efficiency of scheduling algorithm. The details show as follows:

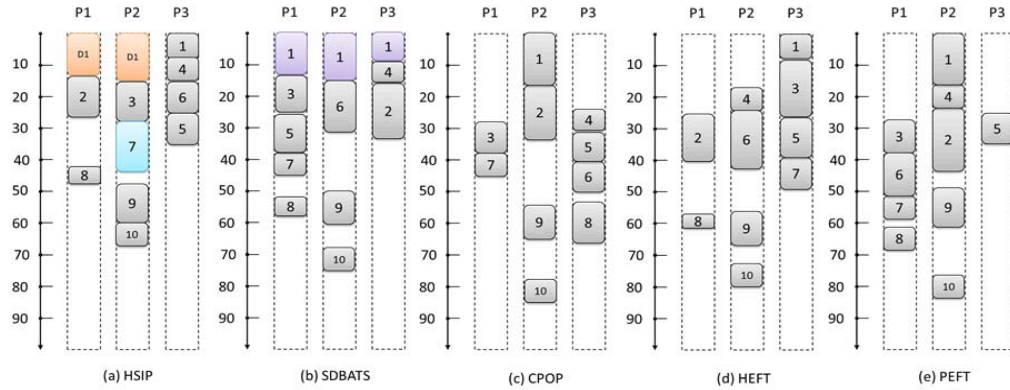


Figure 2. Schedules of the sample task graph in Figure 1 with (a) HSIP, (b) SDBATS, (c) CPOP, (d) HEFT, and (e) PEFT

3.2.1 Entry Task Duplication Selection Policy

Traditional task duplication algorithm, which is possible to get shorter length of scheduling, but is limited by the overload of the processor utilization, mentioned in the literature [2], [5], [7].

For the entry task, because it is the first scheduled task, so there is no need to take processor overload problem into account. Only the entry task needs following duplication selection policy:

1. Choose the processor p_j , which produce the minimum EFT for entry task;
2. Determine whether the entry task in other processor p_i ($p_i \in P$) need to be duplicated. If equation (6) is true, then do Entry Task Duplication in p_i , otherwise, do nothing.

$$w_{v_{entry},i} < w_{v_{entry},j} + c_{v_{entry},v_n} \quad (6)$$

The end condition of the above loop is the first satisfying one from the following two judgments:

1. All processors have been assigned tasks, namely, each processor's entry task duplication judgment has completed;
2. All immediate successor nodes of the entry task (v_n) are scheduled, do not need entry task to transmit data.

3.2.2 ITS Insertion-Based Optimizing Policy

We refine HEFT algorithm's Inserting-based constraints, and provide a choosing policy when there are multiple slots satisfying the conditions. The detailed description is as follows:

1. After completion of task allocation, update each processors' ITS queue;

2. When allocate v_i , look up ITS of all processors to find a slot with $w_{i,j} \leq \text{ITS}$;
3. To all of the ITS meet the condition in step 2, determine when the v_i assigned to the ITS execution, whether the EFT is less than or equal to the lower limit time of ITS;
4. When there are multiple time slots satisfying steps 2 and 3, choose the ITS with smallest EFT.

3.3 Detailed Description of the HSIP Algorithm

The detailed steps of HSIP algorithm are as follows.

Algorithm 1. The HSIP Algorithm.

Input : DAG, set of tasks V , set of Processors P

Output : Schedule result, makespan

1. Starting from the exit node, compute rank_u for all tasks by traversing given DAG upward using equation (5).
 2. Sort the tasks in scheduling list by decreasing order of rank_u value.
 3. **While** there are unscheduled tasks in the list **do**
 4. Select the first task v_i from the list for scheduling
 5. **If** the task is the entry task
 6. Use entry task duplication selection policy as 3.2.1
 7. **Else** (task v_i is not the entry task)
 8. **if** satisfy the condition of ITS insertion-based optimizing policy
 9. Use ITS insertion-based optimizing policy as 3.2.2
 10. **else**
 11. **for** each processor p_j in the processor set ($p_j \in P$) **do**
 12. Compute the earliest finish time (EFT) by equation (3)
 13. **end**
 14. Assign task v_i to the processor p_j that minimize EFT of task v_i
 15. **End if**
 16. **End if**
 17. Update list
 18. **End while**
-

Figure 2 shows the scheduling results for the sample DAG in Figure 1 with the algorithms HSIP, SDBATS, CPOP, HEFT and PEFT. The corresponding result is shown in Table 2, and we can see that HSIP algorithm has shorter makespan and lesser communication costs. It is worth

mentioning that in Figure 2. (a), $D1$ is determined by entry task duplication selection policy of v_{entry} duplication; v_7 is by ITS insertion-based optimizing policy allocation on the processor P_2 , both are the reason of our algorithm can achieve significant scheduling results.

TABLE 2 RESULTS OF SCHEDULING DAG

	HSIP	SDBATS	CPOP	HEFT	PEFT
Task Prioritizing	V1,v2,v4,v3,v6,v5,v9,v7,v8,v10	V1,v3,v4,v2,v6,v5,v7,v9,v8,v10	V1,v2,v3,v7,v4,v5,v9,v6,v8,v10	V1,v3,v2,v4,v5,v6,v9,v7,v8,v10	V1,v4,v2,v5,v3,v6,v7,v9,v8,v10
<u>makespan</u>	67	76	86	80	85

HSIP algorithm has the same time complexity as the HEFT algorithm. Computing ranku (v_i) must traverse all tasks and compare processors which could be done within $O(v, p)$ in initialization phase. Scheduling all tasks must traverse all tasks which could be done in $O(v)$. Computing the EFT of all tasks could be done in $O(v, p)$. Thus, the complexity of the algorithm HSIP is $O(v^2, p)$.

4. EXPERIMENTAL RESULTS AND DISCUSSION

This section compares the performance of the HSIP algorithm with that of the algorithms presented above. For this purpose, we consider use randomly generated application graphs [9]. We first present the comparison metrics used for the performance evaluation.

4.1 Comparison Metrics

The metric most commonly used to evaluate a schedule for a single DAG is the makespan, as defined by (1). Since a large set of task graphs with different properties is used, it is necessary to normalize the schedule length to the lower bound, which is called the Schedule Length Ratio (SLR) [2]. SLR is defined as follows:

$$SLR = \frac{makespan}{\sum_{v_i \in CP_{MIN}} \min_{p_j \in P} (w_{i,j})} \quad (7)$$

In the general case, efficiency is defined as the speedup divided by the number of processors used in each run, and the speedup value is computed by dividing the sequential execution time by the parallel execution time (i.e., the makespan)[5], as shown by the following equation:

$$Speedup = \frac{\min_{p_j \in P} \{\sum_{v_i \in V} w_{i,j}\}}{makespan} \quad (8)$$

4.2 Random Graph Generator

As part of this work we have implemented a random task graph generator that allows the user to generate a variety of test DAGs with various characteristics that depends on several input parameters and they are number of tasks in the graph (v), out degree (β), in degree (γ), shape parameter of a graph (α), Communication to Computation Ratio (CCR) and Range percentage of computation cost (η) [4].

For experiments, we set the following range of values for the parameters. $v = \{20, 40, 60, 80, 100\}$, $\alpha = \{0.5, 1.0, 2.0\}$, $\beta = \{1, 2, 3, 4, 5\}$, $\gamma = \{1, 2, 3, 4, 5\}$, $CCR = \{0.1, 0.5, 1.0, 5.0, 10.0\}$ and $\eta = \{0.1, 0.5, 1.0\}$.

Figure 3 and Figure 4 show the average SLR and average Speedup for varying number of tasks respectively, each data node is the average value given by the number 1125 experiments.

HSIP algorithm has higher performance compared other algorithms; CPOP is the worst because all the critical path tasks are in the same processors. The performance of the algorithm is also evaluated by average SLR with respect to the graph structure, by varying α value with 0.5, 1.0 and 2.0 and the results obtained by this experiment are shown in Figure 5. The results confirm that HSIP algorithm substantially outperforms reported algorithms in terms of average SLR for random task graphs with various shapes.

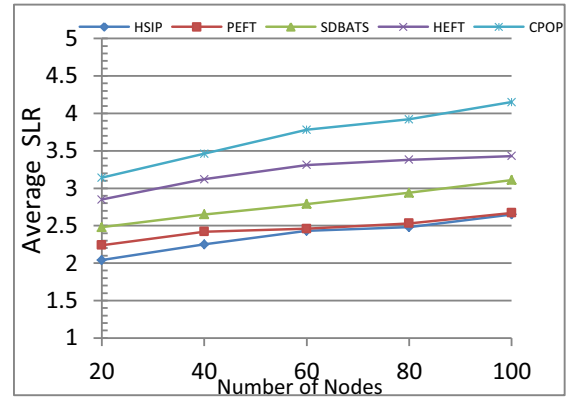


Figure 3. Average SLR for different number of tasks

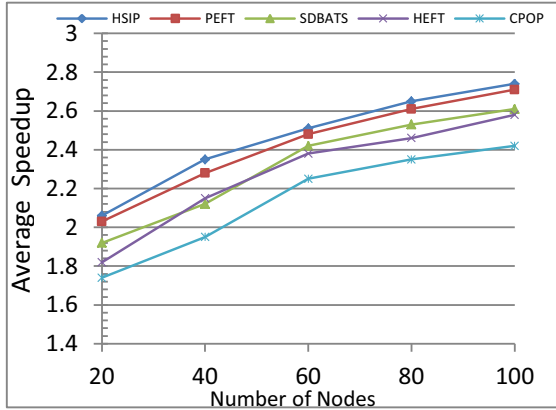


Figure 4. Average Speedup for different number of tasks

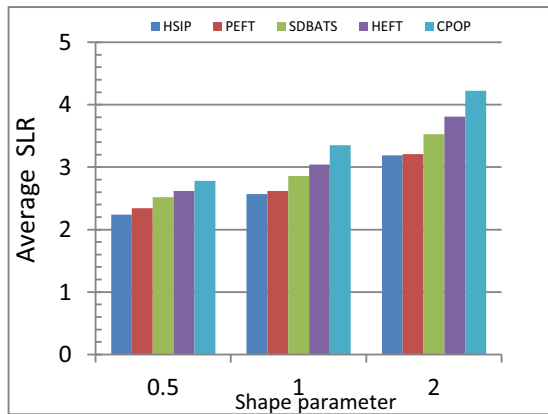


Figure 5. Average SLR for different Shape parameters

5. CONCLUSIONS

In this paper, we proposed a new list scheduling algorithm for heterogeneous systems called HSIP. The task scheduling algorithm proposed here has been proven to be better for scheduling DAG structured applications onto heterogeneous computing system in terms of performance matrices (average schedule length ratio, speedup, and efficiency) and scheduling time. The simulation results confirm that HSIP algorithm is substantially better than the existing algorithms such as PEFT, SDBATS, CPOP and HEFT. All the source code can be provided by email.

ACKNOWLEDGMENT

This work is partially supported by The National Natural Science Foundation of China under Grant No. 11372067 and The General Project of Science and Technology Research from Education Department of Liaoning Province (serial number: L2014508).

REFERENCES

- [1] J D Ullman. NP-complete scheduling problems[J]. Journal of Computer and System sciences, 1975, 10(3): 384-393.
- [2] H Topcuoglu, S Hariri, M Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. Parallel and Distributed Systems, IEEE Transactions on, 2002, 13(3): 260-274.
- [3] E U Munir, S Mohsin, A Hussain, et al. SDBATS: a novel algorithm for task scheduling in heterogeneous computing systems[C]//Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW), 2013 IEEE 27th International. IEEE, 2013: 43-53.
- [4] E Ilavarasan, P Thambidurai, R Mahilmanan. Performance effective task scheduling algorithm for heterogeneous computing system[C]//Parallel and Distributed Computing, 2005. ISPD 2005. The 4th International Symposium on. IEEE, 2005: 28-38.
- [5] H Arabnejad, J G Barbosa. List scheduling algorithm for heterogeneous systems by an optimistic cost table[J]. Parallel and Distributed Systems, IEEE Transactions on, 2014, 25(3): 682-694.
- [6] M I Daoud, N Kharm. A high performance algorithm for static task scheduling in heterogeneous distributed computing systems[J]. Journal of Parallel and distributed computing, 2008, 68(4): 399-409.
- [7] X Tang, K Li, G Liao, et al. List scheduling with duplication for heterogeneous computing systems[J]. Journal of parallel and distributed computing, 2010, 70(4): 323-329.
- [8] E U Munir, J Li, S Shi, et al. A new heuristic for task scheduling in heterogeneous computing environment[J]. Journal of Zhejiang University Science A, 2008, 9(12): 1715-1723.
- [9] R P Dick, D L Rhodes, W Wolf. TGFF: task graphs for free[C]//Proceedings of the 6th international workshop on Hardware/software codesign. IEEE Computer Society, 1998: 97-101.